

FICHE 2 – BASTION GUACAMOLE - DEBIAN 13 – INSTALLATION AVEC DOCKER

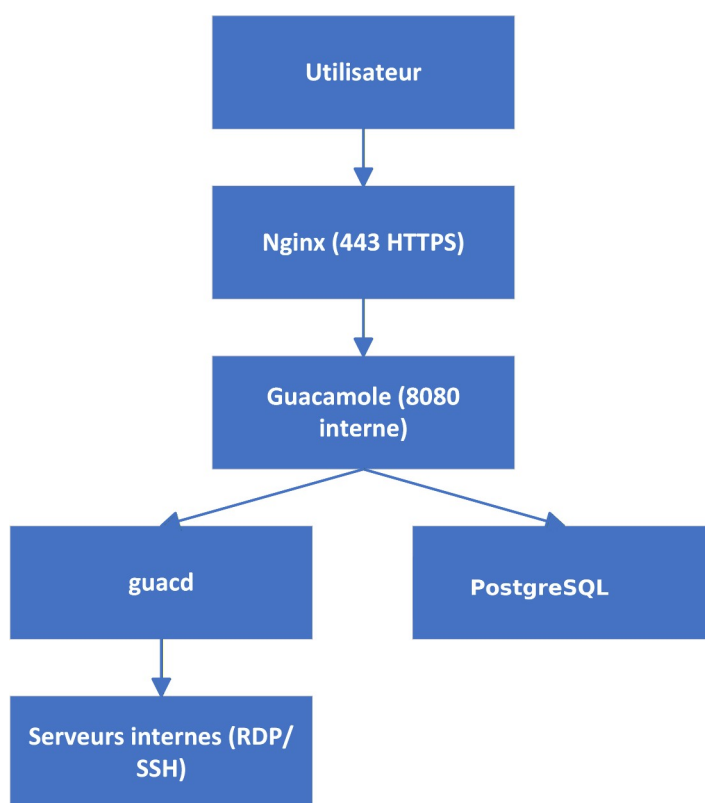
Lien officiel de l'installation : <https://guacamole.apache.org>

Dans le cadre de l'installation du bastion basé sur Apache Guacamole, le choix de Docker peut nous permettre de contourner certaines incompatibilités dues au changement de version du système Debian (exemple : non prise en compte à cette date de freerdp3 par le bastion Guacamole).

L'architecture de Guacamole repose sur trois composants distincts - guacamole (interface web), guacd (proxy RDP/VNC/SSH) et PostgreSQL (base de données).

Docker permet de les exécuter de manière isolée, avec un déploiement rapide, une portabilité facilitée entre environnements et un redémarrage automatique des services en cas d'incident, garantissant ainsi stabilité et maintenabilité du bastion.

Schéma Architecture Guacamole avec Docker



A. INSTALLER ET PARAMÉTRER DOCKER ET LE PARE-FEU UFW

Mise à jour des sources et du système :

```
apt update && apt upgrade -y
```

- **Installation de Docker et Docker Compose v2 :**

```
apt install -y ca-certificates curl gnupg
curl -fsSL https://get.docker.com | sh
```

Activation de Docker au démarrage du système :

```
systemctl enable docker
```

Démarrer Docker immédiatement :

```
systemctl start docker
```

Vérifier l'état de Docker :

```
systemctl status docker
```

Un bastion constitue un point d'entrée sensible. Les communications réseau doivent être limitées au strict nécessaire selon le principe du moindre privilège.

- **Installation du pare-feu UFW :**

```
apt install -y ufw
```

Corriger la variable d'environnement PATH si nécessaire pour que ufw soit trouvé directement par le système :

```
export PATH=$PATH:/usr/sbin
```

Supprimer d'anciennes règles potentiellement dangereuses du pare-feu :

```
ufw reset
```

Définir les politiques par défaut (refuser les connexions entrantes et autoriser les sorties) pour respecter le principe du moindre privilège :

```
ufw default deny incoming
```

```
ufw default allow outgoing
```

Ouvrir les ports essentiels :

```
ufw allow 22/tcp          # SSH administration
```

```
ufw allow 8080/tcp        # HTTP local pour test Guacamole
```

```
ufw allow 8443/tcp        # Accès HTTPS sécurisé via Nginx
```

```
ufw allow 443/tcp         # Accès HTTPS sécurisé
```

Activation du pare-feu UFW pour appliquer les règles :

```
ufw enable
```

Vérifier que UFW est actif et les règles appliquées :

```
ufw status verbose
```

Le pare-feu UFW est maintenant actif et les ports nécessaires sont configurés. Attention : Docker gère également ses propres règles réseau ; seuls les ports Docker strictement nécessaires doivent être publiés.

B. CONFIGURER LE DÉPLOIEMENT DES CONTENEURS POUR LE BASTION GUACAMOLE

Créer un répertoire de travail pour Docker :

```
mkdir -p /opt/guacamole
```

```
cd /opt/guacamole
```

Création d'un dossier « **initdb** » pour l'exécution du script d'initialisation de la base de données PostgreSQL (exemple : création de tables de connexion des utilisateurs, utilisateurs, permissions). La commande rend exécutables les scripts du dossier :

```
mkdir -p ./initdb
```

```
ls -l ./initdb
```

Génération du fichier **initdb.sql** pour PostgreSQL (version Guacamole 1.6.0) :

```
docker run --rm 'guacamole/guacamole:1.6.0' /opt/guacamole/bin/initdb.sh -- postgresql > ./initdb/initdb.sql
```

L'option **–rm** signifie que le conteneur sera supprimé à la fin de l'opération.

Création du fichier **docker-compose.yml** pour déployer les 3 services :

- **guacd (proxy protocolaire)**
- **postgres (base de données)**
- **guacamole (interface web)**

```
services:
# guacd
guacd:
  container_name: guacd_cub
  image: guacamole/guacd:1.6.0
  networks:
    - guacnetwork
  restart: always
  volumes:
    - ./drive:/drive:rw

# postgres
postgres:
  container_name: postgres_cub
  environment:
    PGDATA: /var/lib/postgresql/data/guacamole
    POSTGRES_DB: guacamole
    POSTGRES_PASSWORD: 'MotDePasseRootFort'
    POSTGRES_USER: guacamole
  image: postgres:15.2-alpine
  networks:
    - guacnetwork
  restart: always
  volumes:
    - ./initdb:/docker-entrypoint-initdb.d:z
    - ./data:/var/lib/postgresql/data:Z

# guacamole
guacamole:
  container_name: guacamole_cub
  group_add:
    - "1000"
  depends_on:
    - guacd
    - postgres
  environment:
    GUACD_HOSTNAME: guacd
    POSTGRESQL_ENABLED: "true"
```

```

    REMOTE_IP_VALVE_ENABLED: "true"
    POSTGRESQL_DATABASE: guacamole
    POSTGRESQL_HOSTNAME: postgres
    POSTGRESQL_PASSWORD: 'MotDePasseRootFort'
    POSTGRESQL_USERNAME: guacamole
    image: guacamole/guacamole:1.6.0
    networks:
      - guacnetwork
    ports:
      - 8080:8080/tcp
    restart: always

networks:
  guacnetwork:
    driver: bridge

```

Lancer les conteneurs :

```

docker compose up -d
docker ps

```

Test HTTP local :

- URL : `http://IP_BASTION:8080/guacamole/`
- Login par défaut : **guacadmin / guacadmin**
- À la première connexion, modifier immédiatement le mot de passe du compte guacadmin.

Explications (optionnelles pour les collègues sensibilisés à docker compose) :

Déclaration du service guacd.

C'est le démon proxy utilisé par Guacamole pour gérer les connexions RDP, VNC, SSH.
guacd:

Nom explicite du conteneur (au lieu d'un nom généré automatiquement).
container_name: guacd_cub

Image Docker officielle de Guacamole (composant guacd), version 1.6.0.
image: guacamole/guacd:1.6.0

Connecte ce conteneur au réseau Docker interne **guacnetwork**.
Networks:
- guacnetwork

Redémarre automatiquement le conteneur en cas de crash et au redémarrage du serveur.
restart: always

Monte le dossier local **./drive** dans le conteneur à **/drive** avec droit **lecture/écriture**.
volumes:
- ./drive:/drive:rw

Déclaration du service postgres.

```

# Déclaration du service base de données.
postgres:

# Nom personnalisé du conteneur PostgreSQL.
container_name: postgres_cub
# Variables d'environnement passées au conteneur.
environment:

# Indique où PostgreSQL stockera ses fichiers de données à l'intérieur du conteneur.
PGDATA: /var/lib/postgresql/data/guacamole

# Nom de la base créée automatiquement au premier démarrage.
POSTGRES_DB: guacamole

# Mot de passe du superutilisateur PostgreSQL.
POSTGRES_PASSWORD: 'MotDePasseRootFort'

# Utilisateur principal créé au démarrage.
POSTGRES_USER: guacamole

# Image officielle PostgreSQL version 15.2 basée sur Alpine (plus légère).
image: postgres:15.2-alpine

# Connexion au même réseau interne que les autres services.
Networks:
- guacnetwork

# Redémarrage automatique.
restart: always

# Monte le dossier ./initdb contenant les scripts SQL. Ils seront exécutés automatiquement au
premier démarrage si la base est vide.
Volumes:
- ./initdb:/docker-entrypoint-initdb.d:z

# Stocke physiquement les données PostgreSQL sur l'hôte.
- ./data:/var/lib/postgresql/data:Z

# Déclaration du service guacamole
# Interface web de Guacamole.
guacamole:

# Nom du conteneur.
container_name: guacamole_cub

# Ajoute le conteneur au groupe Linux ID 1000 (utile pour permissions sur volumes partagés).
# group_add inutile ici
#

# Indique que guacd et postgres doivent démarrer avant.
depends_on:
- guacd
- postgres

# Variables d'environnement pour la connexion base de données.
environment:

# Nom du service guacd (résolution DNS interne Docker).

```

```
GUACD_HOSTNAME: guacd
POSTGRESQL_ENABLED: "true"
REMOTE_IP_VALVE_ENABLED: "true"
```

Nom de la base utilisée.

```
POSTGRESQL_DATABASE: guacamole
```

Nom du service PostgreSQL sur le réseau Docker.

```
POSTGRESQL_HOSTNAME: postgres
```

Mot de passe correspondant à celui défini côté PostgreSQL.

```
POSTGRESQL_PASSWORD: 'MotDePasseRootFort'
```

Utilisateur de connexion à la base.

```
POSTGRESQL_USERNAME: guacamole
```

Image officielle Guacamole version 1.6.0.

```
image: guacamole/guacamole:1.6.0
```

Connexion au réseau interne.

```
Networks:
```

```
- guacnetwork
```

Expose le port 8080 du conteneur vers le port 8080 de la machine hôte.

Accès via : http://IP_SERVEUR:8080/guacamole

```
Ports:
```

```
- 8080:8080/tcp
```

Redémarrage automatique.

```
restart: always
```

Déclare un réseau Docker personnalisé :

bridge = réseau interne isolé

Les conteneurs peuvent communiquer entre eux

La base n'est pas exposée publiquement

```
networks:
```

```
guacnetwork:
```

```
driver: bridge
```

C. CONFIGURER LE CHIFFREMENT SSL

Création des dossiers pour stocker les certificats et la configuration de nginx :

```
mkdir -p ./nginx/ssl
```

```
mkdir -p ./nginx/templates
```

Création du certificat auto-signé pour HTTPS :

```
openssl req -nodes -newkey rsa:2048 -new -x509 \
```

```
-keyout nginx/ssl/self-ssl.key \
```

```
-out nginx/ssl/self.cert \
```

```
-subj '/C=FR/ST=Centre-Val-de-Loire/L=Tours/O=CUB/CN=guac.cub.local'
```

Création du fichier de configuration de nginx « guacamole.conf.template » à placer dans le dossier « template » :

```
server {
```

```
listen 443 ssl;
```

```

http2 on;
server_name localhost;

    ssl_certificate /etc/nginx/ssl/self.cert;
    ssl_certificate_key /etc/nginx/ssl/self-ssl.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers on;
    ssl_ciphers "EECDH+AESGCM:EDH+AESGCM:AES256+EECDH:AES256+EDH";
    ssl_ecdh_curve secp384r1;
    ssl_session_cache shared:SSL:10m;
    ssl_session_tickets off;
    ssl_stapling off;
    ssl_stapling_verify off;

location / {
    proxy_pass http://guacamole:8080/guacamole/;
    proxy_buffering off;
    proxy_http_version 1.1;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection $http_connection;
    proxy_cookie_path /guacamole/ /;
    access_log off;
    # allow large uploads (default=1m)
    # 4096m = 4GByte
    client_max_body_size 4096m;
}

#error_page 404 /404.html;

# redirect server error pages to the static page /50x.html
#
error_page 500 502 503 504 /50x.html;
location = /50x.html {
    root /usr/share/nginx/html;
}
}

```

Modifications du fichier docker-compose.yml :

- Arrêt et suppression des conteneurs actuellement actifs :

```
docker compose down
```

- **Modification du port pour le service guacamole :**

```
ports:
## on désactive le port d'écoute initial pour HTTP
##   - 8080:8080/tcp
## le port 8080 reste accessible uniquement dans le réseau Docker
  - 8080/tcp
```

- **Ajout du service nginx :**

```
# nginx
nginx:
  container_name: nginx_cub
  restart: always
  image: nginx:latest
  volumes:
    - ./nginx/templates:/etc/nginx/templates:ro
    - ./nginx/ssl/self.cert:/etc/nginx/ssl/self.cert:ro
    - ./nginx/ssl/self-ssl.key:/etc/nginx/ssl/self-ssl.key:ro
  ports:
    - 8443:443
  networks:
    - guacnetwork
```

- **Création des conteneurs via le nouveau script docker-compose.yml :**

```
docker compose up -d
```

Test HTTPS local :

- URL : `https://IP_BASTION:8443/`
- Login par défaut : **guacadmin / guacadmin**
- À la première connexion, modifier immédiatement le mot de passe du compte guacadmin.